AIML LAB MANUAL

AI-Based Hospital Vacuum Cleaner

Name	Prachi Lohkare
PRN	202401040025
Batch	A1
Division	A
Roll No.	04
Subject	Artificial Intelligence and Machine Learning

Experiment Title: Intelligent Hospital Cleaning Agent

Application: AI-based hospital cleaning and sanitation management.

1. AIM

To design and implement an intelligent hospital cleaning agent in Python that works in different hospital areas, performs area-specific cleaning operations, manages working time and battery, handles waste-bin capacity, and avoids obstacles.

2. THEORY

An intelligent agent observes its environment, makes decisions according to the current condition, and performs an appropriate action. In this application, the agent is a hospital cleaning robot rather than a conventional vacuum cleaner.

The hospital environment contains different areas such as the Emergency Room, ICU, Operation Theatre, General Ward, Waiting Area, Corridor, Washroom and Pharmacy. Each area can require a different cleaning operation.

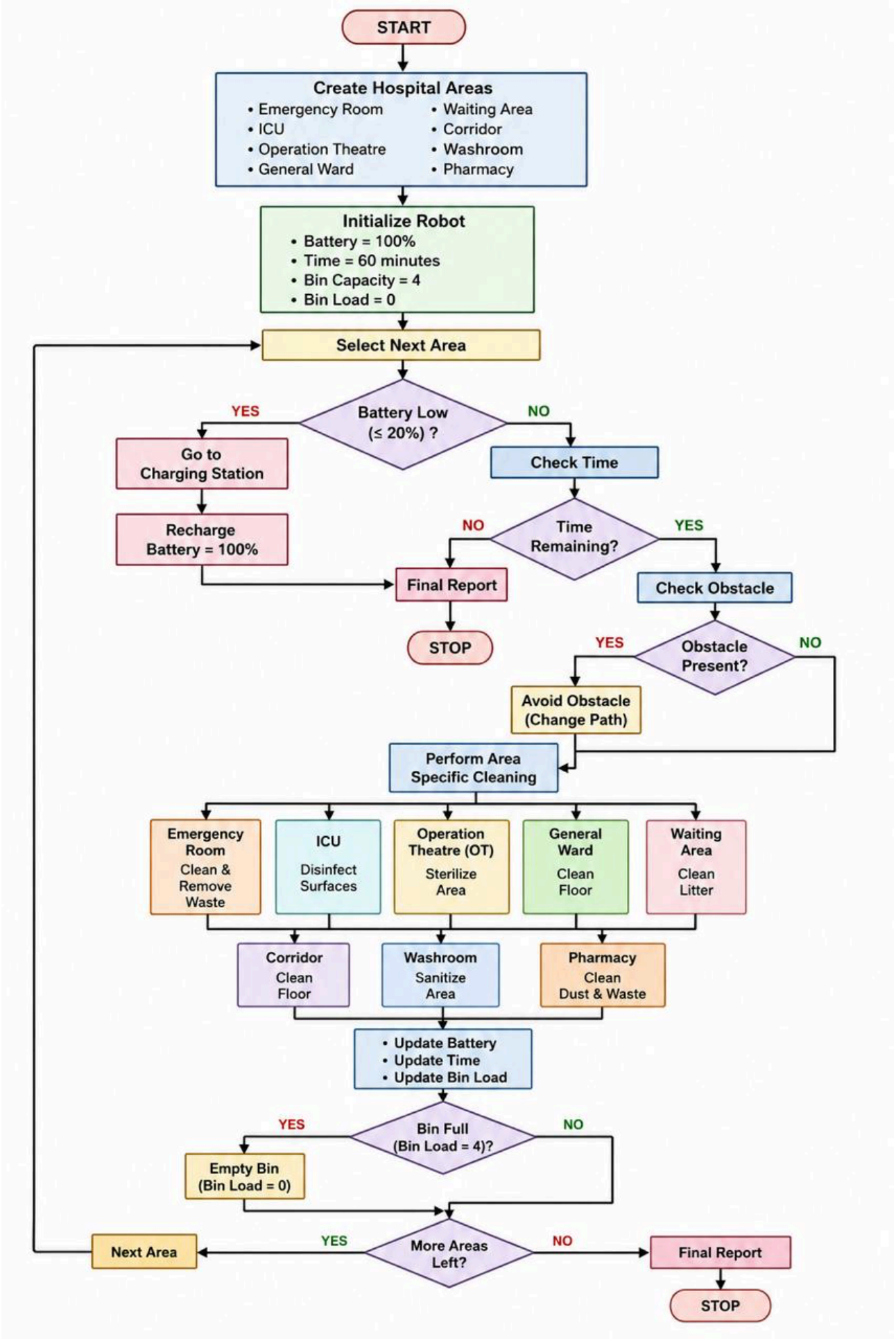
The robot uses simple rule-based decisions. When an area needs cleaning, the robot calls the function designed for that area. The robot also checks its battery, working time, waste-bin capacity and obstacles.

- Emergency Room → cleaning and waste removal
- ICU → surface disinfection
- Operation Theatre → sterilization
- General Ward → floor and waste cleaning
- Waiting Area → surface and litter cleaning
- Corridor → floor cleaning
- Washroom → sanitation
- Pharmacy → dust and waste cleaning

3. PEAS

Component	Description
Performance	Areas cleaned, obstacles avoided, battery remaining, working time remaining and waste-bin condition.
Environment	Emergency Room, ICU, Operation Theatre, General Ward, Waiting Area, Corridor, Washroom and Pharmacy.
Actuators	Cleaning mechanism, disinfecting/sterilizing mechanism, waste collection unit, movement system and charging connection.
Sensors / Percepts	Area condition, obstacle presence, battery level, working time and bin load.

4. FLOWCHART


START
Create Hospital Areas
• Emergency Room
• ICU
• Operation Theatre
• General Ward
• Waiting Area
• Corridor
• Washroom
• Pharmacy
Initialize Robot
• Battery = 100%
• Time = 60 minutes
• Bin Capacity = 4
• Bin Load = 0
Select Next Area
Battery Low (≤ 20%) ?
YES
NO
Go to Charging Station
Recharge Battery = 100%
Check Time
Time Remaining?
NO
YES
Final Report
STOP
Check Obstacle
Obstacle Present?
YES
NO
Avoid Obstacle (Change Path)
Perform Area Specific Cleaning
Emergency Room
Clean & Remove Waste
ICU
Disinfect Surfaces
Operation Theatre (OT)
Sterilize Area
General Ward
Clean Floor
Waiting Area
Clean Litter
Corridor
Clean Floor
Washroom
Sanitize Area
Pharmacy
Clean Dust & Waste
• Update Battery
• Update Time
• Update Bin Load
Bin Full (Bin Load = 4)?
YES
NO
Empty Bin (Bin Load = 0)
More Areas Left?
YES
NO
Next Area
Final Report
STOP

4. OBSERVATION

The exact output changes because the initial cleaning condition and obstacle status are generated randomly. A representative observation is shown below.

Parameter	Example Observation
Emergency Room	Needs Cleaning
ICU	Needs Cleaning
Operation Theatre	Clean
General Ward	Needs Cleaning
Waiting Area	Clean
Corridor	Needs Cleaning
Washroom	Needs Cleaning
Pharmacy	Clean
Initial Battery	100%
Working Time	60 minutes
Bin Capacity	4 units

5. CODE

```
import random


# Creates the hospital environment
class Hospital:


    def __init__(self):


        # Areas covered by the cleaning robot
        self.areas = [
            "Emergency Room",
            "ICU",
            "Operation Theatre",
            "General Ward",
            "Waiting Area",
            "Corridor",
            "Washroom",
            "Pharmacy"
        ]


        # Randomly decide which areas need cleaning
        self.condition = {
            area: random.choice(["Needs Cleaning", "Clean"])
            for area in self.areas
        }


        # Random obstacles like beds or wheelchairs
```

```
        self.obstacles = {
            area: random.choice([True, False])
            for area in self.areas
        }


# AI hospital cleaning robot
class HospitalCleaningRobot:

    def __init__(self, hospital):

        self.hospital = hospital

        # Robot's starting conditions
        self.battery = 100
        self.time = 60

        # Waste carrying capacity
        self.bin_capacity = 4
        self.bin_load = 0

        # Keep track of work completed
        self.cleaned_areas = 0
        self.obstacles_avoided = 0
        self.recharges = 0

    # Recharge the robot when the battery is low
```

```
def recharge(self):

    print("Battery is low.")
    print("Returning to charging station...")

    self.battery = 100
    self.recharges += 1

    print("Robot fully recharged.")

# Check battery before starting work
def check_battery(self):

    if self.battery <= 20:
        self.recharge()

# Empty the waste bin when it is full
def check_bin(self):

    if self.bin_load >= self.bin_capacity:

        print("Waste bin is full.")
        print("Emptying waste bin.")

        self.bin_load = 0

# Avoid beds, wheelchairs or other obstacles
```

```
def check_obstacle(self, area):

    if self.hospital.obstacles[area]:

        print("Obstacle found.")
        print("Robot changes its path.")

        self.obstacles_avoided += 1
        self.battery -= 5
        self.time -= 2

# Clean emergency room
def clean_emergency(self):

    print("Cleaning emergency room.")
    print("Removing waste and cleaning floor.")

    self.time -= 8
    self.battery -= 10
    self.bin_load += 1

# Disinfect ICU
def clean_icu(self):

    print("Disinfecting ICU surfaces.")

    self.time -= 10
```

```
    self.battery -= 12
    self.bin_load += 1

# Sterilize operation theatre
def sterilize_ot(self):

    print("Sterilizing operation theatre.")

    self.time -= 12
    self.battery -= 15
    self.bin_load += 1

# Clean general ward
def clean_ward(self):

    print("Cleaning general ward.")

    self.time -= 6
    self.battery -= 8
    self.bin_load += 1

# Clean waiting area
def clean_waiting_area(self):

    print("Cleaning waiting area.")

    self.time -= 5
```

```
    self.battery -= 7
    self.bin_load += 1

# Clean hospital corridor
def clean_corridor(self):

    print("Cleaning corridor.")

    self.time -= 4
    self.battery -= 5

# Sanitize washroom
def clean_washroom(self):

    print("Sanitizing washroom.")

    self.time -= 7
    self.battery -= 9
    self.bin_load += 1

# Clean pharmacy
def clean_pharmacy(self):

    print("Cleaning pharmacy.")

    self.time -= 5
    self.battery -= 6
```

```
    self.bin_load += 1

# Start the hospital cleaning process
def start(self):

    print("\n================================")
    print("    AI HOSPITAL CLEANING ROBOT")
    print("================================")

    for area in self.hospital.areas:

        print("\nArea:", area)

        # Stop if working time is finished
        if self.time <= 0:
            print("Working time finished.")
            break

        # Recharge before working if needed
        self.check_battery()

        # Check for obstacles
        self.check_obstacle(area)

        # Clean only if the area needs cleaning
        if self.hospital.condition[area] == "Needs Cleaning":
```

```
if area == "Emergency Room":
    self.clean_emergency()

elif area == "ICU":
    self.clean_icu()

elif area == "Operation Theatre":
    self.sterilize_ot()

elif area == "General Ward":
    self.clean_ward()

elif area == "Waiting Area":
    self.clean_waiting_area()

elif area == "Corridor":
    self.clean_corridor()

elif area == "Washroom":
    self.clean_washroom()

elif area == "Pharmacy":
    self.clean_pharmacy()

# Mark the area as cleaned
self.hospital.condition[area] = "Clean"
self.cleaned_areas += 1
```

```
            # Check bin after collecting waste
            self.check_bin()

        else:

            print("Area is already clean.")

        print("Battery:", self.battery, "%")
        print("Time remaining:", self.time, "minutes")
        print("Bin:", self.bin_load, "/", self.bin_capacity)

    self.report()

# Display final result
def report(self):

    print("\n========== FINAL REPORT ==========")

    print("Areas Cleaned     :", self.cleaned_areas)
    print("Obstacles Avoided :", self.obstacles_avoided)
    print("Recharges         :", self.recharges)
    print("Battery Remaining :", self.battery, "%")
    print("Time Remaining    :", self.time, "minutes")
    print(
        "Bin Load          :",
        self.bin_load,
```

```
            "/",
            self.bin_capacity
        )


# Create hospital
hospital = Hospital()

# Create cleaning robot
robot = HospitalCleaningRobot(hospital)

# Start cleaning
robot.start()
```

6. RESULT ANALYSIS

- The program creates a hospital environment containing multiple functional areas.
- The robot performs different cleaning operations according to the selected hospital area.
- Battery level is checked and the robot recharges when the battery becomes low.
- Working time limits the duration of the cleaning process.
- Obstacles such as beds and wheelchairs are handled by changing the robot's path.
- Waste-bin capacity is monitored and the bin is emptied when it becomes full.
- The final report shows cleaned areas, obstacles avoided, recharges, remaining battery, remaining time and bin load.

7. RESULT

The AI-Based Hospital Cleaning Robot was successfully implemented in Python. The agent performs area-specific cleaning operations, manages battery and working time, handles waste-bin capacity, avoids obstacles and produces a final performance report.

8. CONCLUSION

The experiment demonstrates the working of an intelligent agent in a hospital environment. Instead of using the same cleaning action everywhere, the robot performs different functions for the Emergency Room, ICU, Operation Theatre, General Ward, Waiting Area, Corridor, Washroom and Pharmacy. Battery management, time management, bin capacity and obstacle handling make the simulated system more realistic and suitable for an AI practical.

9. GIT LINK

https://github.com/prachilohkare8-debug/AIML-202401040025